



Guia do Aluno — O Padrão MVC (e o restaurante)

UC: Programação Back-End | **Bloco 2** — Aula de 26/08/2026

Projeto: API de Gestão da Livraria (SA1)

O que você vai conseguir fazer ao final

- ☐ Explicar o que é MVC usando a analogia do restaurante
- ☐ Rodar o primeiro servidor Express da Livraria
- ☐ Criar a estrutura de pastas `routes`, `controllers`, `services`, `models`
- ☐ Saber qual pergunta fazer para decidir onde um código deve morar

Onde trabalhar: hoje é tudo no **repositório do grupo** (`livraria-api-grupoN`).

```
cd livraria-api-grupoN
git pull
```

1. O restaurante sem organização

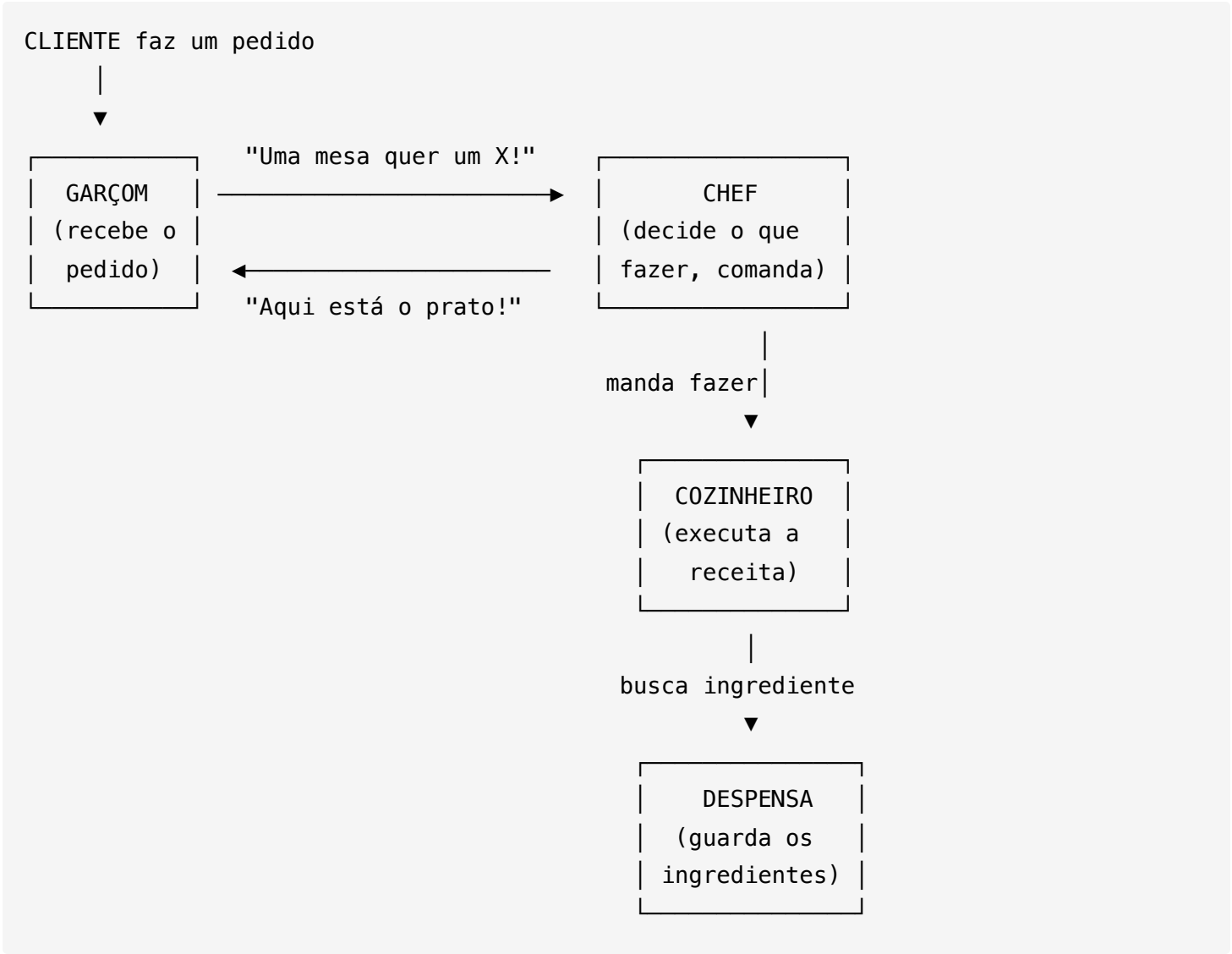
Imagine um restaurante sem garçom, sem chef, sem despensa separada. O cliente entra, vai direto para a cozinha, mexe nas panelas, procura os ingredientes sozinho, tenta cozinhar o próprio prato brigando com outro cliente que também está tentando cozinhar o dele.

Funciona? Talvez, com pouca gente. Com o restaurante cheio, é caos.

É exatamente isso que acontece quando todo o código de uma API fica em um arquivo só: dado, cálculo, resposta, tudo misturado. Funciona no começo. Quando o projeto cresce, vira bagunça — e ninguém sabe mais onde mexer sem quebrar outra coisa.

2. O restaurante organizado — MVC

Um restaurante que funciona bem divide o trabalho em **4 papéis** claros:



Cada papel tem **uma função só**, e não invade a função do outro:

Papel no restaurante	Nome em código	O que faz
Garçom	ROTA	Recebe o pedido e leva para quem sabe fazer
Chef	CONTROLLER	Decide o que precisa acontecer, comanda a cozinha
Cozinheiro	SERVICE	Executa a receita de verdade — calcula, prepara
Despensa	MODEL	Guarda os ingredientes (os dados) organizados

💡 **Frase para não esquecer:** *"Quem decide não é quem executa."*

MVC é a sigla de três dessas camadas: **Model**, **View/Controller** — em API de back-end como a nossa, não existe "View" (tela pronta), então usamos **Controller** e adicionamos **Service** no meio, formando o que costuma se chamar de **MVC com camada de service**.

📌 MVC é um dos vários **"padrões de projeto"** (design patterns) que existem — moldes prontos para resolver problemas comuns de organização de código. Vocês vão ouvir esse termo de novo mais para frente no curso.

🔄 **A tabela acima é a última vez que o restaurante aparece como protagonista.** A partir da seção 3, a explicação passa a ser sempre com a Livraria — o restaurante só volta entre parênteses, como lembrete rápido.

3. Cada papel, agora na Livraria

A partir daqui a explicação é sempre em cima da Livraria — o nome do restaurante aparece só entre parênteses, como lembrete, não como uma segunda história.

👤 **Rota — a porta de entrada**

A **rota** é a porta de entrada da API: ela recebe a requisição (por exemplo, `GET /livros`) e só encaminha para o controller certo. **Sem lógica dentro dela** — ela não decide nada sobre livros.

(o "garçom" — recebe, não decide)

👨‍🍳 **Controller — quem decide o que fazer**

O **controller** recebe o que veio da rota e decide **o que precisa acontecer** para atender aquele pedido — buscar todos os livros? um só? Ele chama o service certo e devolve a resposta pronta.

(o "chef" — decide, comanda)

🔪 **Service — quem executa de verdade**

O **service** guarda a lógica de negócio de verdade: buscar, calcular, validar. É onde mora o **"como fazer"** — separado do "o que fazer", que já foi decidido pelo controller.

(o "cozinheiro" — executa)

Model — onde os dados moram

O **model** é onde os dados moram. E aqui vai uma notícia boa: **vocês já criaram models sem saber que eram models.** `Livro`, `Categoria`, `Pessoa`, `Cliente`, `Funcionario` — todas as classes que vocês escreveram desde 05/08 **são** models. Hoje elas só ganham um endereço fixo: a pasta `src/models/`.

(a "despensa" — guarda os dados)

4. O fluxo completo, na Livraria

Um pedido "quero ver todos os livros" percorre as 4 camadas assim:

"Quero ver todos os livros"

Cliente (navegador) → ROTA (GET /livros)

→ CONTROLLER (o que fazer com esse pedido?)

→ SERVICE (busca e organiza os livros)

→ MODEL (onde os livros estão guardados)

← devolve os livros prontos

← controller devolve a resposta

Cliente recebe a lista de livros

Repare: a informação vai **descendo** (rota → controller → service → model) e depois **sobe de volta** até chegar ao cliente. Cada camada só conversa com a vizinha, nunca pula direto para a que está duas casas abaixo.

💡 Se travar em algum momento, volte à frase curta: **"quem decide não é quem executa."**

5. O primeiro servidor Express

Até hoje, todo código de vocês rodava e terminava sozinho. Hoje muda: vamos criar um programa que **fica esperando** pedidos, como um restaurante de portas abertas.

Instalar o Express

```
npm init -y
npm install express
```

💡 Repare: **sem** `-D` ou `--save-dev`. O Express vai para `dependencies`, porque diferente do Nodemon e do ESLint, ele é necessário para a API rodar de verdade — inclusive depois de pronta.

O Hello World da Livraria

Substitua o conteúdo do `src/index.js` do grupo por:

```
const express = require("express");

const app = express();
const PORTA = 3000;

app.get("/", (req, res) => {
  res.send("API da Livraria no ar!");
});

app.listen(PORTA, () => {
  console.log("Servidor rodando em http://localhost:" + PORTA);
});
```

Rode:

```
npm run dev
```

Abra o navegador em <http://localhost:3000> — a mensagem deve aparecer na tela.

⚠️ **O terminal não vai voltar para você digitar outro comando.** O servidor fica rodando, esperando pedidos, como um restaurante de portas abertas esperando clientes. Para parar: `Ctrl + C`.

O que cada linha faz:

Linha	O que é
<code>const express = require("express")</code>	Traz a biblioteca instalada
<code>const app = express()</code>	Cria a aplicação — o "restaurante" propriamente dito
<code>app.get("/", (req, res) => {...})</code>	Define uma rota: quando alguém pedir <code>/</code> , faça isso
<code>req</code>	O pedido que chegou (request)
<code>res</code>	A resposta que você vai devolver (response)
<code>res.send(...)</code>	Envia a resposta de volta para quem pediu
<code>app.listen(PORTA, ...)</code>	Liga o restaurante — começa a escutar pedidos na porta 3000

😬 Essa única rota hoje faz o papel de garçom, chef e cozinheiro ao mesmo tempo — porque é só uma linha. Daqui a pouco a gente separa isso de verdade em pastas.

6. A estrutura de pastas

Dentro de `src/`, criar as pastas que faltam:

```
cd src
mkdir routes controllers services
```

`models/` já existe desde a atividade de 07/08 — confira que ela está lá.

A estrutura final fica assim:

```
livraria-api-grupoN/
├─ src/
│   ├── index.js           ← o servidor (o "restaurante" ligado)
│   ├── routes/            ← os garçons
│   ├── controllers/       ← os chefs
│   ├── services/          ← os cozinheiros
│   └─ models/             ← a despensa (Livro, Categoria, Pessoa...)
```

Um arquivo "placa na porta" em cada pasta nova

Ainda não vamos escrever a lógica de verdade — isso é conteúdo do Bloco 3, quando o banco de dados entrar. Hoje, cada arquivo só **declara sua função**, com um comentário:

src/routes/livroRoutes.js

```
// ROTA = o garcom.  
// Aqui vao ficar os caminhos (endpoints) relacionados a Livro.  
// Ex: GET /livros, POST /livros  
// Implementacao chega no Bloco 3, quando o banco de dados entrar.  
  
module.exports = {};
```

src/controllers/livroController.js

```
// CONTROLLER = o chef.  
// Aqui vai ficar a decisao do que fazer com cada pedido de Livro.  
// Recebe da rota, chama o service certo, devolve a resposta.  
// Implementacao chega no Bloco 3.  
  
module.exports = {};
```

src/services/livroService.js

```
// SERVICE = o cozinheiro.  
// Aqui vai ficar a logica de verdade: buscar, calcular, validar.  
// Implementacao chega no Bloco 3.  
  
module.exports = {};
```

💡 **Por que criar arquivo vazio, se ele não faz nada ainda?** Porque hoje o objetivo **é a estrutura**, não o funcionamento. É como construir a planta do restaurante — as salas existem, com placa na porta, mesmo antes do primeiro cliente entrar.

7. Consolidando os models do grupo

Cada integrante fez sua própria versão de `Livro.js` e `Categoria.js`, no repositório pessoal. Hoje o grupo escolhe **uma versão** de cada e coloca dentro de `src/models/` do repositório do grupo — o mesmo já foi feito com `Pessoa`, `Cliente` e `Funcionario` em 07/08.

Não precisa ser "a versão perfeita" — pode ser a mais fácil de todo mundo entender, já que o grupo inteiro vai trabalhar nela dali para frente.

Checklist

- ☐ `express` instalado, listado em `dependencies` no `package.json`
- ☐ `src/index.js` roda com `npm run dev` e mostra a mensagem em `http://localhost:3000`
- ☐ Pastas `routes/`, `controllers/`, `services/` criadas dentro de `src/`
- ☐ `models/` conferida — já existia desde 07/08
- ☐ Um arquivo comentado em cada pasta nova (`livroRoutes.js`, `livroController.js`, `livroService.js`)
- ☐ `Livro.js` e `Categoria.js` consolidados em `src/models/`, com a versão escolhida pelo grupo
- ☐ Consigo explicar, com minhas palavras, o que cada camada faz
- ☐ `git push` feito

Erros comuns

Sintoma	O que aconteceu	Como resolver
<code>Cannot find module 'express'</code>	Esqueceu de rodar <code>npm install express</code>	Rode o comando dentro da pasta do repositório do grupo
A página do navegador não carrega	O servidor não está rodando	Confira se o <code>npm run dev</code> ainda está ativo no terminal
O terminal "travou"	Não travou — o servidor está esperando pedidos	Isso é normal. <code>Ctrl + C</code> para parar

Sintoma	O que aconteceu	Como resolver
EADDRINUSE: address already in use	Já existe outro servidor rodando na porta 3000	Feche o terminal anterior, ou troque PORTA para outro número, como 3001
Página mostra erro em vez da mensagem	Algum erro de digitação no src/index.js	Leia a mensagem de erro no terminal — ela aponta a linha

Cola rápida

```
#Iniciar um projeto
npm init -y

# Instalar Express
npm install express

# Rodar o servidor
npm run dev

# Parar o servidor
Ctrl + C

# Criar as pastas do esqueleto MVC
cd src
mkdir routes controllers services
```

ROTA	(garçom)	→ recebe o pedido
CONTROLLER	(chef)	→ decide o que fazer
SERVICE	(cozinheiro)	→ executa a receita
MODEL	(despensa)	→ guarda os dados